SESIÓN 02: SERVIDORES Y ROLES DE ADMINISTRACIÓN

Base de datos Magdalena · MySQL / phpMyAdmin

1. ¿De qué trata esta sesión?

En esta sesión se aprende a administrar un servidor de base de datos creando **diferentes usuarios con distintos niveles de permisos**. No todas las personas que trabajan con una base de datos deben tener el mismo acceso. La idea principal es el **principio de mínimo privilegio**: cada usuario recibe solo los permisos necesarios para realizar su trabajo, ni más ni menos.

Esto ayuda a mantener la seguridad de la información y evita que alguien modifique o elimine datos importantes por error o de forma malintencionada.

Introducción

No conviene que todas las personas que tocan la base de datos tengan el mismo poder. Quien solo genera reportes no debería poder borrar tablas. Esta práctica separa tres perfiles en MySQL para la base Magdalena_ramos.

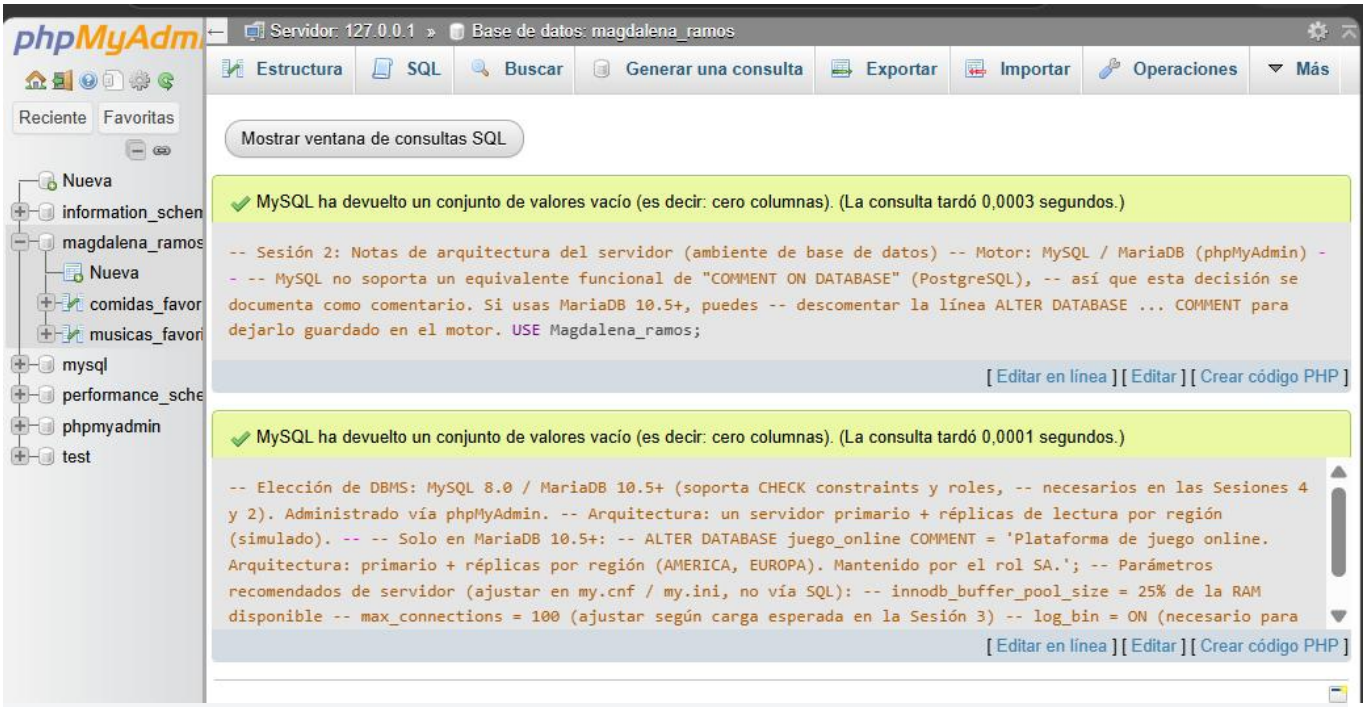
- DBA: autoridad completa sobre Magdalena_ramos (crear, modificar, borrar y otorgar privilegios).
- SA: supervisa el motor (procesos, recargas, réplicas) sin leer datos de clientes ni inventario.
- DA: solo lectura sobre tablas de catálogo elegidas (estadísticas sin riesgo de alterar datos).

Paso 1 · Arquitectura del servidor

Este bloque documenta decisiones de diseño (motor, réplicas, parámetros). USE Magdalena_ramos; indica al servidor con qué base trabajar. Las líneas con -- son comentarios.

```
USE Magdalena_ramos;

-- Arquitectura (comentarios ejecutables)
-- Motor: MySQL 8.0 / MariaDB 10.5+
-- Diseño: primario + réplicas de lectura por región (simulado)
-- Parámetros de referencia (my.cnf / my.ini):
--   innodb_buffer_pool_size ≈ 25% RAM
--   max_connections = 100
--   log_bin = ON
```



Qué hace: selecciona el esquema activo. No modifica tablas; solo deja constancia escrita de la arquitectura.

Paso 2 · Crear usuarios DBA, SA y DA

Se registran tres cuentas. Cada una recibe únicamente los privilegios de su función.

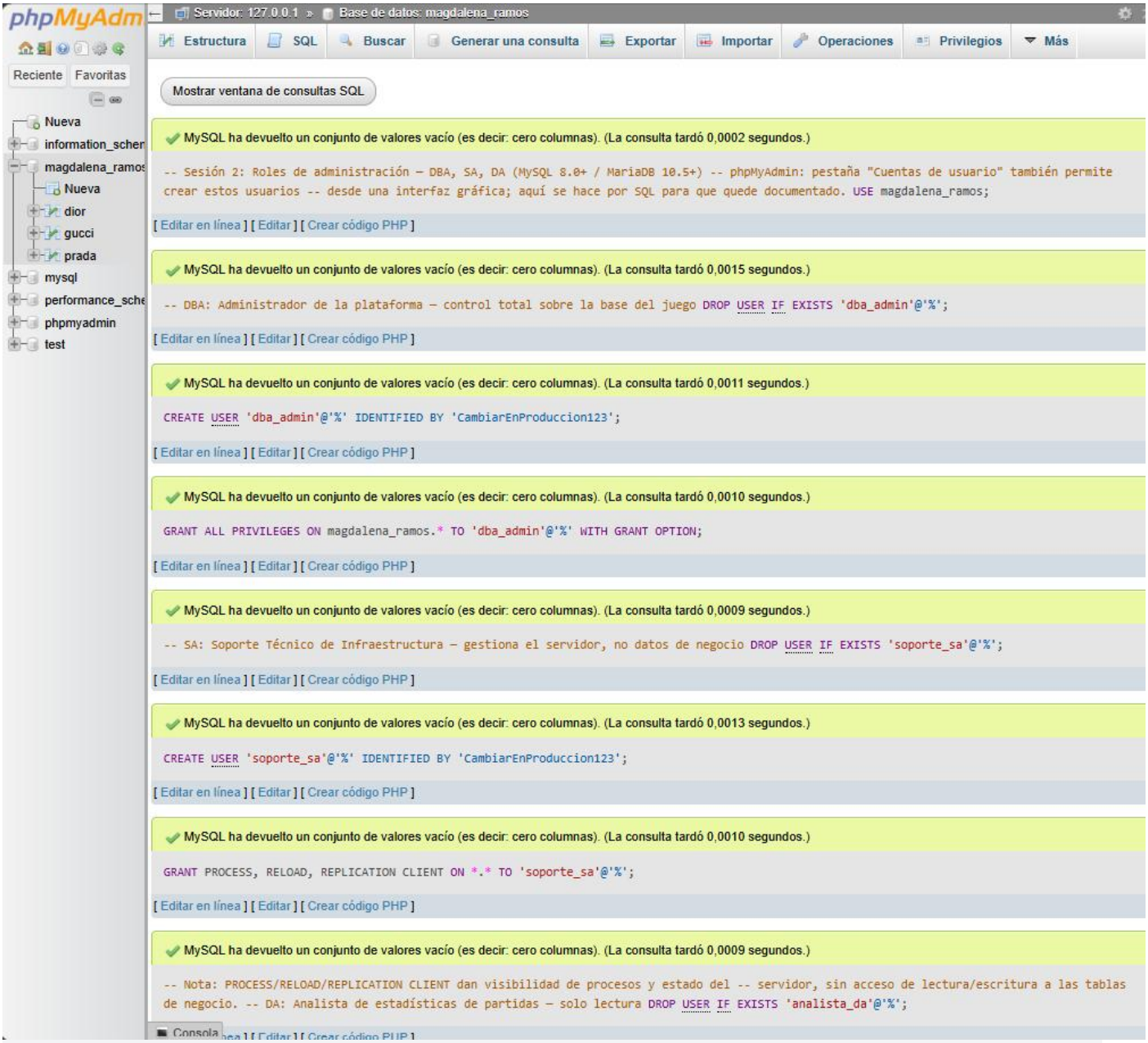
```
USE magdalena_ramos;

-- DBA: control total
DROP USER IF EXISTS 'dba_admin'@'%';
CREATE USER 'dba_admin'@'%' IDENTIFIED BY 'CambiarEnProduccion123';
GRANT ALL PRIVILEGES ON magdalena_ramos.* TO 'dba_admin'@'%' WITH GRANT OPTION;

-- SA: solo operaciones de servidor
DROP USER IF EXISTS 'soporte_sa'@'%';
CREATE USER 'soporte_sa'@'%' IDENTIFIED BY 'CambiarEnProduccion123';
GRANT PROCESS, RELOAD, REPLICATION CLIENT ON *.* TO 'soporte_sa'@'%';

-- DA: solo SELECT sobre catálogo
DROP USER IF EXISTS 'analista_da'@'%';
CREATE USER 'analista_da'@'%' IDENTIFIED BY 'CambiarEnProduccion123';
GRANT SELECT ON magdalena_ramos.dior TO 'analista_da'@'%';
GRANT SELECT ON magdalena_ramos.gucci TO 'analista_da'@'%';
GRANT SELECT ON magdalena_ramos.prada TO 'analista_da'@'%';

FLUSH PRIVILEGES;
```



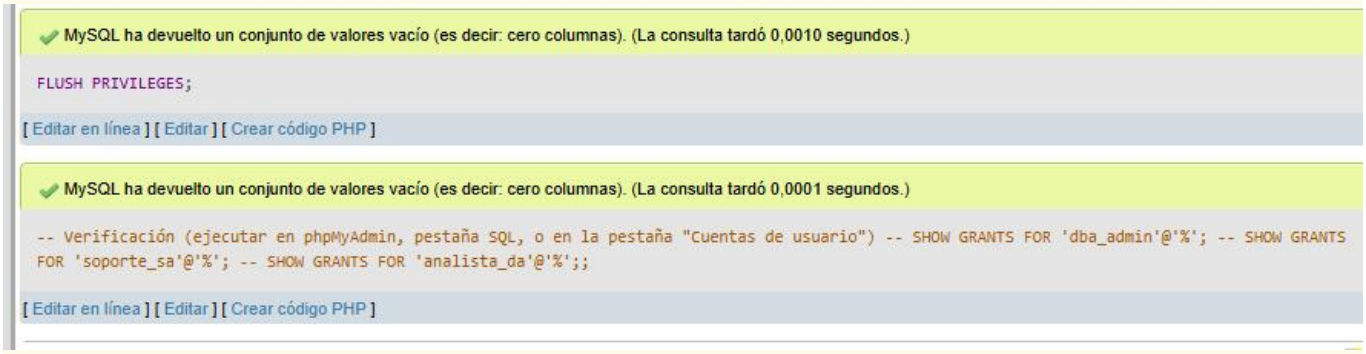
Explicación breve de los comandos

- DROP USER IF EXISTS: borra la cuenta si ya existía (el script se puede repetir).
- CREATE USER ... IDENTIFIED BY: crea la cuenta con contraseña temporal.
- GRANT ALL ... WITH GRANT OPTION (DBA): privilegios totales y capacidad de cederlos.
- GRANT PROCESS, RELOAD, REPLICATION CLIENT (SA): ver procesos y estado del servidor; sin acceso a filas de negocio.
- GRANT SELECT ON tabla (DA): lectura exclusiva de las tablas indicadas.
- FLUSH PRIVILEGES: aplica los cambios de inmediato.

Verificación

```
SHOW GRANTS FOR 'dba_admin'@'%';
SHOW GRANTS FOR 'soporte_sa'@'%';
SHOW GRANTS FOR 'analista_da'@'%';
```

Criterio de éxito: analista_da solo debe mostrar SELECT (nunca INSERT, UPDATE o DELETE).



6. Qué entendí de la Sesión 02

El tema principal es la **administración y creación de diferentes roles de usuarios**. Primero se revisa la arquitectura del servidor y luego se crean tres usuarios:

- **DBA** es el administrador y tiene todos los permisos sobre la base de datos.
- **SA** se encarga de funciones relacionadas con el servidor, pero no puede acceder a los datos de los clientes/jugadores.
- **DA** es un analista que solamente puede consultar algunas tablas (Jugadores, Partidas y Logros).

Para asignar los permisos se utiliza el comando **GRANT**. También se usan **CREATE USER** para crear usuarios y **DROP USER** para eliminarlos si existen.

La idea más importante es que **cada usuario debe tener solamente los permisos necesarios para realizar su trabajo**. Esto ayuda a mantener la seguridad de la información y evitar modificaciones o accesos que no correspondan.

7. Finalidad de separar los roles

La finalidad es evitar que una persona tenga permisos innecesarios y pueda modificar o eliminar información importante. En el mundo real, normas de seguridad y cumplimiento (como PCI-DSS o SOX) exigen esta separación de funciones para reducir el riesgo de errores y fraudes.

Conclusión de la sesión

La idea clave es el principio de mínimo privilegio: cada cuenta recibe solo lo indispensable. Así se limita el daño por error humano o por una cuenta comprometida. CREATE USER y GRANT son las herramientas para aplicar esa separación en MySQL.